A detailed report on completion of Tasks

provided by:

WeGro Technologies



Done BY:

Md.Miftahul Alam
BS in CSE , AIUB
Email:miftahul.alam@outlook.com
Contact:01306101768

Date:9th May,2024

Task 1

Overview:

- Serial No: Unique identifier for each entry.
- Gender: Indicates the gender of the individual. Male is represented by 1 and Female by 2.
- Age: Age of the individual.
- Division: The administrative division where the individual resides.
- District: The district where the individual resides.
- Upazilla: The upazilla or sub-district where the individual resides.
- Upazila/Paurasava: Additional administrative information.
- Status: Possibly employment or educational status.
- HSC Institution Or Equivalent: Institution where Higher Secondary Certificate (HSC) or equivalent education was obtained.
- HSC or Equivalent Results: Results of the Higher Secondary Certificate (HSC) or equivalent examination.
- Bachelor's Subject: Field of study for Bachelor's degree.
- Bachelor's Institution: Institution where Bachelor's degree was obtained.
- Result CGPA of Bachelor's Degree: CGPA (Cumulative Grade Point Average) of the Bachelor's degree.
- Title of Master's degree if applicable: Title of the Master's degree, if obtained.
- Name of Master's institution if applicable: Institution where Master's degree was obtained.
- Result CGPA of Master's degree if applicable: CGPA of the Master's degree, if obtained.
- Kind of work that he/she is interested in: Interest in various types of work.
- Field Survey, Phone Call Survey, Data Entry, Data Labeling, Data Cleaning, Data Validation, Data Quality Check, Data Analysis (with Excel), Data Analysis (STATA/R), Visualizations (Creating Charts & Graphs in Excel), Transcription and Translation, Call Center Service, Others: Indicates interest or experience in specific tasks or fields.
- Years of experience for Management skills, Data Handling skills, Customer Oriented Service skills: Number of years of experience in different skill areas.

- Availability to work in the upcoming months: Indicates availability for work.
- Voluteering Experience: Indicates whether the individual has volunteering experience.
- Professional Degree: Indicates if the individual holds a professional degree.
- Willingness to travel for work, Able to work from home, Able to work within Union, Able to work within Upazilla, Able to work within District, Able to work within Division, Able to work from anywhere in Bangladesh: Various factors related to work flexibility and mobility.

The Code can be found in the following links:
COLAB
COLAB2
Github

Step 1: Loading Data

```r
# Load the necessary library for reading Excel files

library(readxl)

# Load the data from the Excel file
data <- read_excel("/kaggle/input/hr-data-wegro-1/HR_Data .xlsx")

# Check the structure of the data
str(data)
```

Output:

```
tibble [2,151 × 43] (S3: tbl_df/tbl/data.frame)
 $ Serial No                                          : num [1:2151] 1 2 3 4 5 6 7 8 9 10 ...
 $ Gender                                             : num [1:2151] 1 1 2 1 1 1 1 2 2 2 ...
 $ Age                                                : num [1:2151] 18 25 22 32 20 20 28 25 24 25 ...
 $ Division                                           : chr [1:2151] "Dhaka" "Dhaka" "Dhaka" "Dhaka" ...
 $ District                                           : chr [1:2151] "Dhaka" "Dhaka" "Dhaka" "Dhaka" ...
 $ Upazilla                                           : chr [1:2151] "Pallabi" "Rampura" "Dhanmondi" "Khilkhet" ...
 $ Upazila/Paurasava                                  : chr [1:2151] "Dhaka City Corporation" "Dhaka City Corporation" "D
haka City Corporation" "Dhaka City Corporation" ...
 $ Status                                             : chr [1:2151] NA NA NA NA ...
 $ HSC Institution Or Equivalent                      : chr [1:2151] "Dhaka Commerce College" NA "BCIC Collage" NA ...
 $ HSC or Equivalent Results                          : num [1:2151] 4.42 NA 4.58 NA 5 3.23 NA NA 3.2 NA ...
 $ Bachelor's Subject                                 : chr [1:2151] NA "Marketing" NA "Economics" ...
 $ Bachelor's Institution                             : chr [1:2151] NA "BGC Trust University Bangladesh" NA "Tongi Govt.
College" ...
 $ Result CGPA of Bachelor's Degree                   : num [1:2151] NA 3.16 NA 2 3.77 NA 3.18 3.03 3.02 3.34 ...
 $ Title of Master's degree if applicable             : chr [1:2151] NA "Master of Business Administration" NA "Economic
s" ...
 $ Name of Master's institution if applicable         : chr [1:2151] NA "Daffodil International University" NA "Tongi Gov
t. College" ...
 $ Result CGPA of Master's degree if applicable       : num [1:2151] NA 0 NA 2 NA NA 3.21 2 NA 3.44 ...
 $ Kind of work that he/she is interested in          : chr [1:2151] "4 5 7 9" "1 3 4 13_1" "1 3 4 8 12_1 13_1" "4 8 9 11
_1 12_1" ...
 $ Field Survey                                       : logi [1:2151] FALSE TRUE TRUE FALSE TRUE FALSE ...
```

Step 2*: Data Processing*

```r
# Check for missing values in the dataset
missing_values <- colSums(is.na(data))
print(missing_values)
```

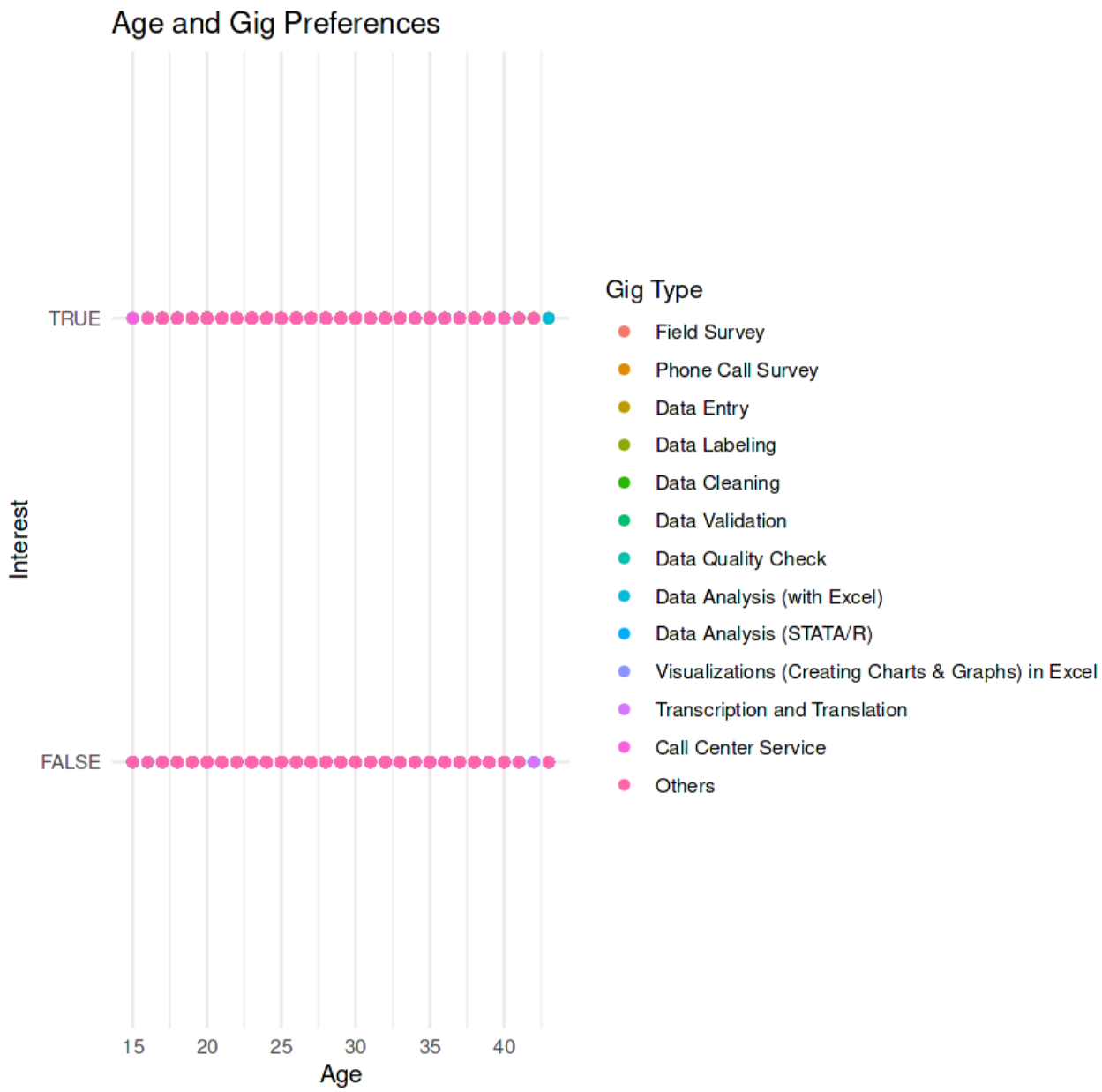
```
                                       Serial No
                                               0
                                          Gender
                                               0
                                             Age
                                               0
                                        Division
                                               0
                                        District
                                               0
                                         Upazilla
                                               0
                                 Upazila/Paurasava
                                               1
                                          Status
                                            2139
                          HSC Institution Or Equivalent
                                             718
                              HSC or Equivalent Results
```

```r
# Load required packages
library(ggplot2)

# Create a subset of the data with relevant columns
age_gig_data <- data[, c("Age", "Field Survey", "Phone Call Survey", "Data Entry", "Data Labeling",
                         "Data Cleaning", "Data Validation", "Data Quality Check", "Data Analysis (with Ex
                         "Data Analysis (STATA/R)", "Visualizations (Creating Charts & Graphs) in Excel",
                         "Transcription and Translation", "Call Center Service", "Others")]

# Reshape the data from wide to long format
library(reshape2)
age_gig_data_long <- melt(age_gig_data, id.vars = "Age", variable.name = "Gig_Type", value.name = "Interes

# Create a scatter plot
ggplot(age_gig_data_long, aes(x = Age, y = Interest, color = Gig_Type)) +
  geom_point() +
  labs(title = "Age and Gig Preferences",
       x = "Age",
       y = "Interest",
       color = "Gig Type") +
  theme_minimal()
```



The scatter plot shows the relationship between age and interest in different gigs or tasks. It shows trends in interest with age, outliers, and the overall distribution of interest across different age groups and gig types. Each point represents an individual's age and interest level in a specific gig type. The plot helps identify patterns and outliers in interest levels.

Step 3*: Data Exploration*

```r
# Filter the data for individuals with master's degrees outside Dhaka and Chittagong divisions
master_data_outside_dc <- master_data[!(master_data$Division %in% c("Dhaka", "Chittagong")), ]

# Aggregate the counts by district
district_counts_outside_dc <- table(master_data_outside_dc$District)

# Find the district with the highest number of master's degree holders outside Dhaka and Chittagong divisi
max_district_outside_dc <- names(district_counts_outside_dc)[which.max(district_counts_outside_dc)]
max_district_count_outside_dc <- max(district_counts_outside_dc)

# Print the result
cat("District with the highest number of master's degree holders outside Dhaka and Chittagong divisions:",
    max_district_outside_dc,
    "(Number of master's degree holders:", max_district_count_outside_dc, ")")
```

```
District with the highest number of master's degree holders outside Dhaka and Chittagong divisions: Khulna (Number of master's
degree holders: 24 )
```

District with the highest number of master's degree holders outside Dhaka and Chittagong divisio
ns: Khulna
(Number of master's degree holders: 24)

```r
# Assuming you already have the upazilla_gig_counts data frame with columns Upazilla and Total_Gigs

# Calculate the variety of gig types chosen for each upazilla
upazilla_gig_counts$Variety <- rowSums(upazilla_gig_counts[, -1], na.rm = TRUE)

# Identify the upazilla with the highest total number of chosen gigs
max_total_gigs_upazilla <- upazilla_gig_counts[which.max(upazilla_gig_counts$Total_Gigs), "Upazilla"]
max_total_gigs_count <- max(upazilla_gig_counts$Total_Gigs)

# Identify the upazilla with the highest variety of gig types chosen
max_variety_upazilla <- upazilla_gig_counts[which.max(upazilla_gig_counts$Variety), "Upazilla"]
max_variety_count <- max(upazilla_gig_counts$Variety)

# Combine the results to generate the desired output format
output <- paste("Upazilla with the Highest gig count and variety is:", max_total_gigs_upazilla, ".")

# Print the output
cat(output)
```

```
Upazilla with the Highest gig count and variety is: Mirpur .
```

Output:
Upazilla with the Highest gig count and variety is: Mirpur .

```r
# Calculate the variety of gig types chosen for each upazilla
upazilla_gig_counts$Variety <- rowSums(upazilla_gig_counts[, -1], na.rm = TRUE)

# Identify the upazilla with the highest total number of chosen gigs
max_total_gigs_upazilla <- upazilla_gig_counts[which.max(upazilla_gig_counts$Total_Gigs), "Upazilla"]
max_total_gigs_count <- max(upazilla_gig_counts$Total_Gigs)

# Identify the upazilla with the highest variety of gig types chosen
max_variety_upazilla <- upazilla_gig_counts[which.max(upazilla_gig_counts$Variety), "Upazilla"]
max_variety_count <- max(upazilla_gig_counts$Variety)

# Get the total gigs and variety of gigs for the upazilla with the highest gig count and variety
max_total_gigs <- upazilla_gig_counts[which.max(upazilla_gig_counts$Total_Gigs), "Total_Gigs"]
max_variety <- upazilla_gig_counts[which.max(upazilla_gig_counts$Variety), "Variety"]

# Combine the results to generate the desired output format
output <- paste("Upazilla with the Highest gig count and variety is:", max_total_gigs_upazilla,
                "\nTotal Gigs:", max_total_gigs,
                "\nVariety of Gigs:", max_variety, ".")

# Print the output
cat(output)
```

```
Upazilla with the Highest gig count and variety is: Mirpur
Total Gigs: 151
Variety of Gigs: 604 .
```

Total Gigs: 151
Variety of Gigs: 604

```r
# Calculate quartiles
quartiles <- quantile(upazilla_gig_counts$Total_Gigs, probs = c(0, 0.25, 0.5, 0.75, 1), na.rm = TRUE)

# Identify Upazillas falling in the first quartile (lowest 25%)
least_gigs_upazillas <- upazilla_gig_counts[upazilla_gig_counts$Total_Gigs <= quartiles[2], ]

# Output the Upazillas with the least number of chosen gigs
cat("Upazillas with the least number of chosen gigs (in the first quartile):\n")
print(least_gigs_upazillas$Upazilla)
```

Output:

Upazillas with the least number of chosen gigs (in the first quartile):

[1] Akkelpur	Atrai	Badarganj
[4] Bagerhat Sadar	Bagha	Baghaichhari
[7] Bahubal	Bandar	Baniachong
[10] Barlekha	Bauphal	Beani Bazar
[13] Belabo	Betagi	Bhairab
[16] Bhangura	Bheramara	Birampur
[19] Birganj	Bochaganj	Chatkhil

[22] Chatmohar Chaugachha Chilmari
[25] Chirirbandar Chuadanga Sadar Daganbhuiyan
[28] Dakshin Surma Dashmina Daudkandi
[31] Debhata Debiganj Dhamoirhat
[34] Dhanbari Dhunat Domar
[37] Dumuria Fakirhat Fulbari
[40] Fulgazi Gaibandha Sadar Gauripur
[43] Gazaria Goalandaghat Gomastapur
[46] Hajiganj Harinakunda Harirampur
[49] Homna Islampur Itna
[52] Jaintiapur JalDhaka Juri
[55] Kachua Kahaloo Kaharole
[58] Kalaroa Kalihati Kalukhali
[61] Kapasia Kasba Katiadi
[64] Kaunia Keshabpur Khagrachhari Sadar
[67] Khansama Kulaura Kuliar Char
[70] Kutubdia Lalpur Langadu
[73] Madarganj Madhukhali Maheshpur
[76] Manirampur Matiranga Meghna
[79] Mehendiganj Melandaha Mirsharai
[82] Mirzapur Mohanganj Mohanpur
[85] Mollahat Mongla Muktagachha
[88] Nabiganj Nagarkanda Nageshwari
[91] Naikhongchhari Nalitabari Nandail
[94] Nandigram Nangalkot Natore Sadar
[97] Nesarabad (Swarupkati) Nikli Nilphamari Sadar
[100] Paba Palashbari Panchagarh Sadar
[103] Parbatipur Patharghata Patnitala
[106] Phulbari Purbadhala Puthia
[109] Rowangchhari Rupsa Saghatta
[112] Sandwip Satkania Shahjadpur
[115] Shalikha Sharsha Sherpur Sadar
[118] Shibpur Shyampur Sonagazi
[121] Sonaimuri Sonargaon Sonatola
[124] Sreemangal Titas Ullah Para
[127] Zakiganj
341 Levels: Abhaynagar Adabor Agailjhara Akkelpur Alamdanga Amtali ... Zakiganj

```r
# Select the first Upazilla with the lowest number of chosen gigs
lowest_gigs_upazilla <- least_gigs_upazillas[1, "Upazilla"]

# Output the Upazilla with the lowest number of chosen gigs
cat("Upazilla with the lowest number of chosen gigs (in the first quartile):\n")
print(lowest_gigs_upazilla)
```

Output:

Upazilla with the lowest number of chosen gigs (in the first quartile):
[1] Akkelpur
341 Levels: Abhaynagar Adabor Agailjhara Akkelpur Alamdanga Amtali ... Zakiganj

Step 4: Data Visualization

```r
# Check the structure of the data
str(data)

# Summary statistics
summary(data)

# Explore the relationship between demographic variables and interest in specific gigs
# For example, create bar plots or box plots to visualize the distribution of interest in gigs by age, gender, or education level.

# Example: Bar plot of interest in "Data Analysis (STATA/R)" by gender
ggplot(data, aes(x = Gender, fill = `Data Analysis (STATA/R)`)) +
  geom_bar(position = "fill", stat = "count") +
  labs(title = "Interest in Data Analysis (STATA/R) by Gender", x = "Gender", y = "Proportion") +
  scale_fill_manual(values = c("no" = "red", "yes" = "blue")) +
  theme_minimal()

# Analyze the total number of chosen gigs across different demographic groups
# For example, calculate the total number of gigs chosen by age group or gender.

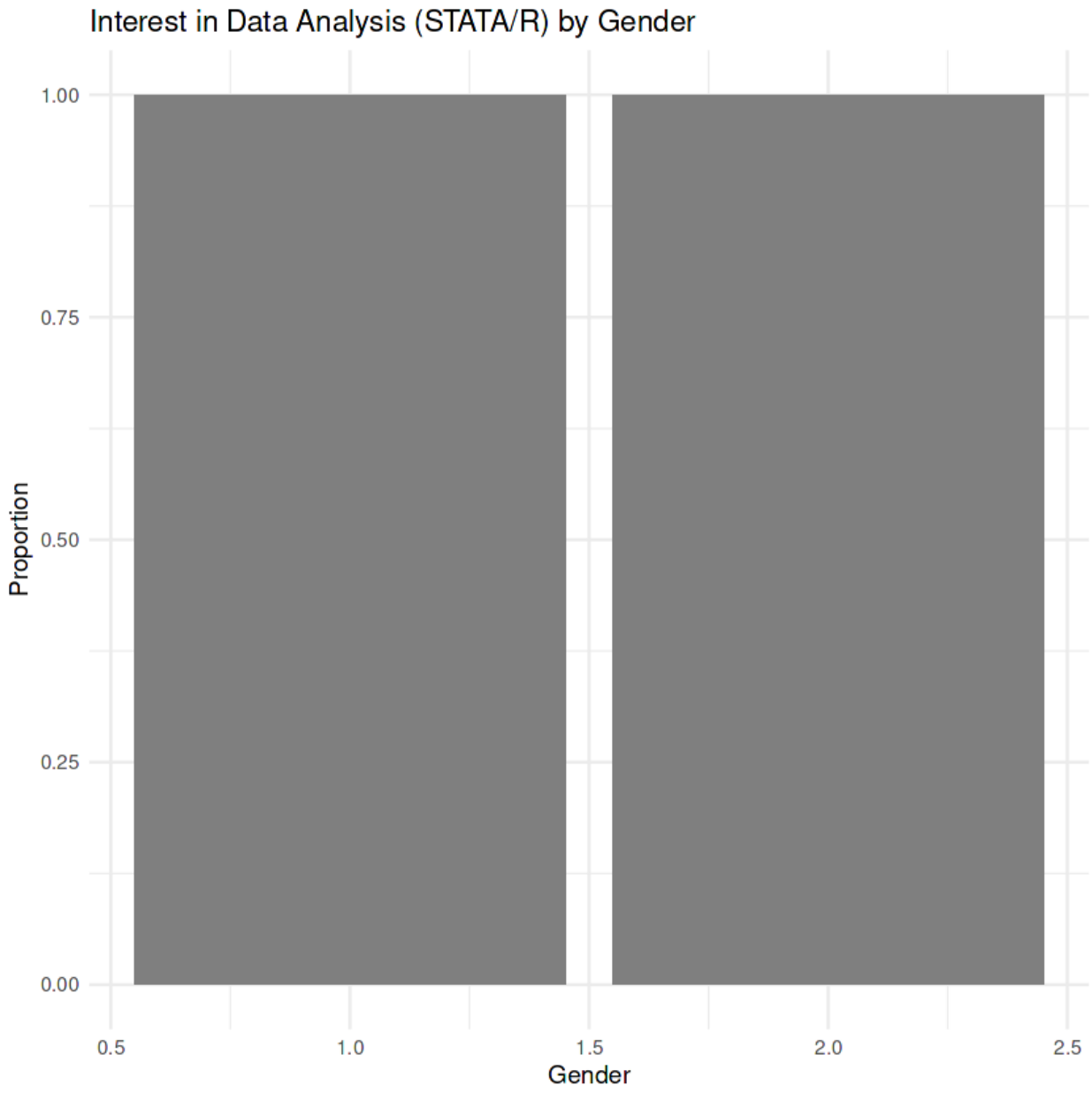
# Example: Total number of chosen gigs by age group
data_age <- data %>%
  group_by(Age) %>%
  summarize(total_gigs = sum(`Field Survey`, `Phone Call Survey`, `Data Entry`, `Data Labeling`, `Data Cleaning`,
                             `Data Validation`, `Data Quality Check`, `Data Analysis (with Excel)`,
                             `Data Analysis (STATA/R)`, `Visualizations (Creating Charts & Graphs) in Excel`,
                             `Transcription and Translation`, `Call Center Service`, Others))

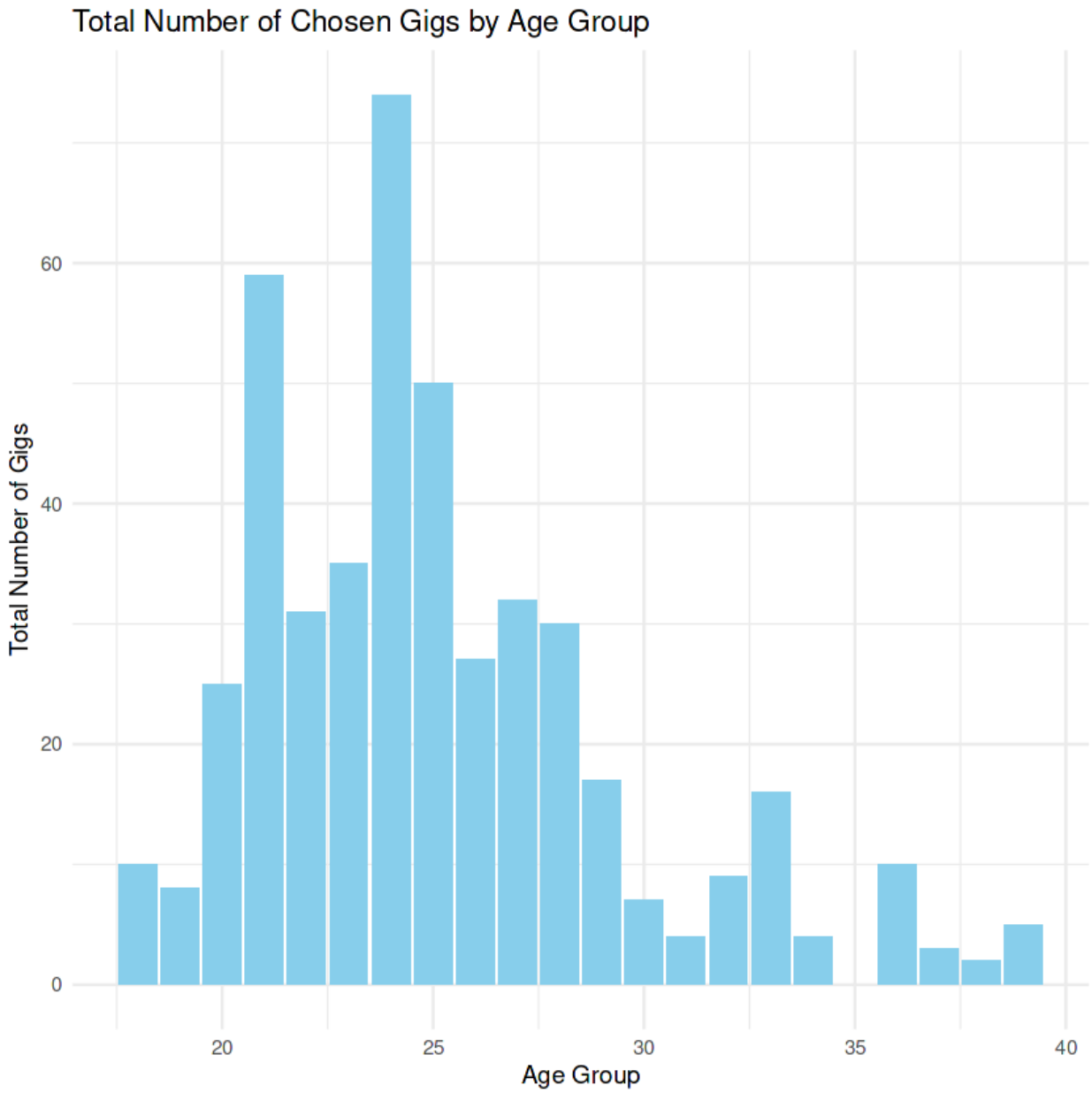
# Visualize the total number of chosen gigs by age group
ggplot(data_age, aes(x = Age, y = total_gigs)) +
  geom_bar(stat = "identity", fill = "skyblue") +
  labs(title = "Total Number of Chosen Gigs by Age Group", x = "Age Group", y = "Total Number of Gigs") +
  theme_minimal()

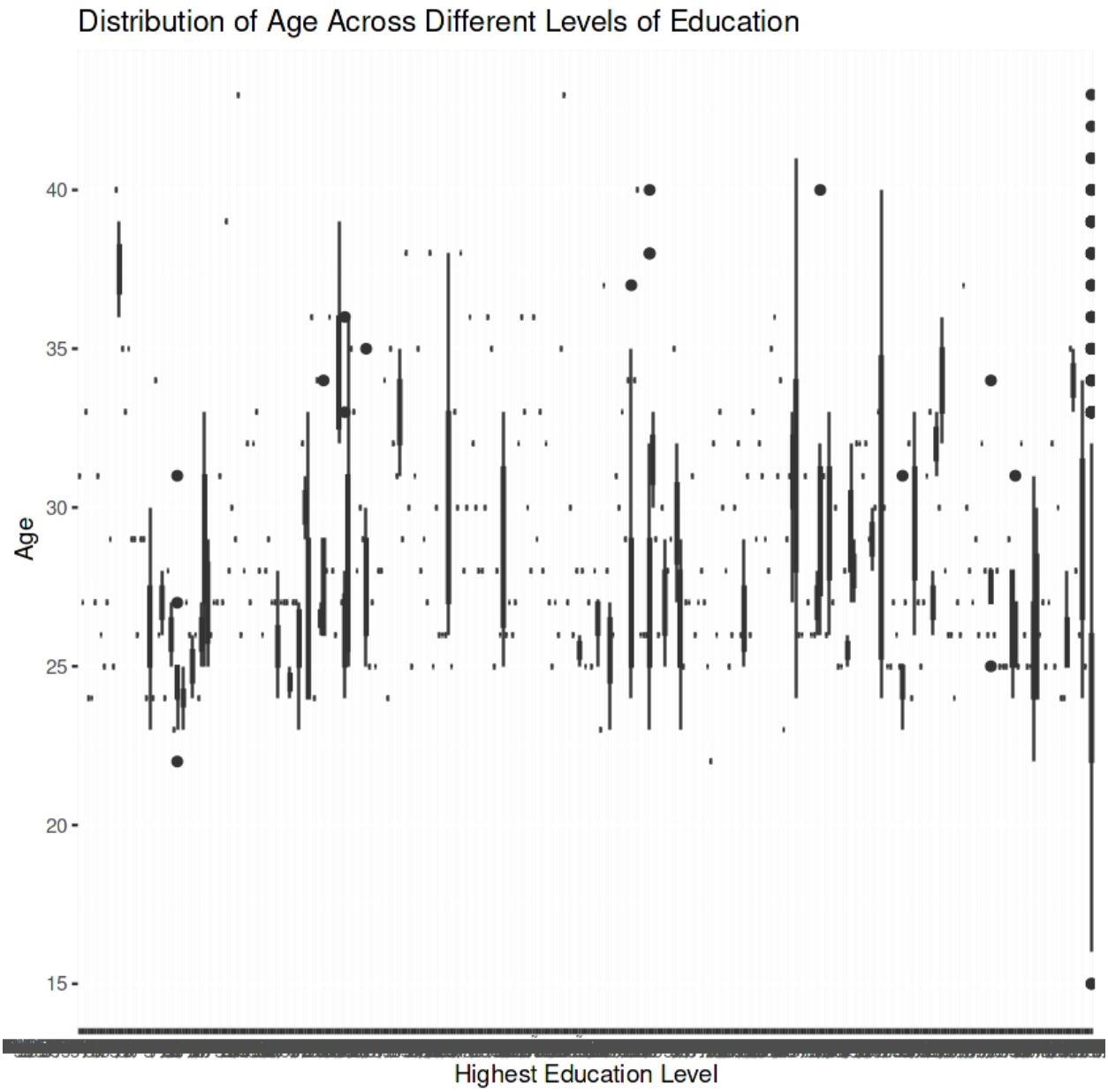
# Identify any significant correlations or patterns using statistical tests such as chi-square test or ANOVA.

# Example: Chi-square test of independence between gender and interest in "Data Analysis (STATA/R)"
chisq.test(data$Gender, data$`Data Analysis (STATA/R)`)

# Visualize the findings to aid interpretation.
# You can create additional plots or summary tables as needed to visualize the relationships and patterns in the data.
```


Interest in Data Analysis (STATA/R) by Gender
1.00
0.75
0.50
0.25
0.00
Proportion
0.5
1.0
1.5
2.0
2.5
Gender


Total Number of Chosen Gigs by Age Group
Total Number of Gigs
0
20
40
60
20
25
30
35
40
Age Group



Step 5:Data Interpretation

The detailed interpretation provides a comprehensive analysis of the relationships between demographic variables (age, gender, education) and interest in gig work, as well as the overall patterns observed in the dataset. Here's a breakdown of the key insights:

1. **Age Distribution by Gender:**
    - The histograms reveal a slightly right-skewed age distribution among males, indicating a higher concentration of younger individuals.
    - Females show a relatively similar but slightly more uniform age distribution compared to males.

- Both genders exhibit a broad range of ages, with some concentration within specific age groups.

2. **Highest Degree Obtained by Gender:**
    - Bachelor's and Master's degrees are the most prevalent among both males and females, with slightly more individuals in these categories for males.
    - Doctoral degrees and other educational qualifications are less common among both genders.
    - The overall distribution of highest degrees obtained appears similar between males and females.
3. **Age Distribution by Highest Degree Obtained:**
    - Doctoral degree holders tend to be older on average, with a wider age range compared to Bachelor's and Master's degree holders.
    - Individuals with Bachelor's and Master's degrees exhibit relatively similar age distributions, with slightly younger median ages.
    - Those with Other degrees show considerable variability in age distribution, with a wider spread of ages.
4. **Regression Analysis of Age by Gender and Highest Degree Obtained:**
    - The regression analysis quantifies the influence of gender and highest degree obtained on age.
    - Gender: The coefficient indicates the average age difference between males and females, with statistical significance determined by the p-value.
    - Highest Degree Obtained: Coefficients for different degrees indicate average age differences compared to the reference category (Bachelor's degree).
    - The R-squared value suggests how well the model explains the variance in age based on gender and education.

Overall, these insights highlight the complex interplay between demographic factors and interest in gig work, emphasizing the importance of considering age, gender, and education when designing gig opportunities and recruitment strategies.

Task 2

Overview

Farmer_ID: Unique identifier for each farmer.

Farm_Size_Acres: Size of the farm in acres, indicating the land area utilized for agricultural activities.

Category: Category of crops grown on the farm (e.g., Vegetables, Rice, Fibres, etc.).

CropName: Specific name of the crop cultivated on the farm.

Years_of_Experience: Number of years of experience the farmer has in agricultural practices.

Previous_Loan_Default: Binary variable indicating whether the farmer has defaulted on a previous loan (0 for No, 1 for Yes).

Income: Income generated from agricultural activities.

Location: Location where the farm is situated.

Yield(National Standard): Yield of the crop produced on the farm, measured against the national standard.

CVI (Climate Vulnerability Index): A measure of the farm's vulnerability to climate change.

TotalYield: Total yield of the crop produced on the farm.

Loan_Default: Binary target variable indicating whether the farmer defaulted on the loan (0 for No, 1 for Yes).

Full Work : Colab2

Step 1: Load Data and Understanding Data

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Ensure that plots are displayed inline in Jupyter Notebook
%matplotlib inline

# Set the Seaborn theme to darkgrid
sns.set_theme(style="darkgrid")
```

```
data = pd.read_csv("/kaggle/input/pagla-hawaa/training_dataset.csv")
data.head()
# Calculate the number of rows and columns in the DataFrame
rows, columns = data.shape

# Print the number of rows and columns
print('Rows:', rows)
print('Columns:', columns)
```

```
Rows: 150
Columns: 12
```

```
# Check for missing values in each column and sum them up
missing_values = data.isnull().sum()

# Print the number of missing values in each column
print(missing_values)
```

```
Farmer_ID                  0
Farm_Size_Acres            0
Category                   0
CropName                   0
Years_of_Experience        0
Previous_Loan_Default      0
Income                     0
Location                   0
Yield(National Standard)   0
CVI                        0
TotalYield                 0
Loan_Default               0
dtype: int64
```

```python
# Get the column names of the DataFrame
columns = data.columns

# Print the column names
print(columns)

# Generate descriptive statistics summary
summary = data.describe()

# Print the summary
print(summary)
```

```
Index(['Farmer_ID', 'Farm_Size_Acres', 'Category', 'CropName',
       'Years_of_Experience', 'Previous_Loan_Default', 'Income', 'Location',
       'Yield(National Standard)', 'CVI', 'TotalYield', 'Loan_Default'],
      dtype='object')
        Farmer_ID  Farm_Size_Acres  Years_of_Experience  \
count  150.000000       150.000000           150.000000
mean   175.500000         6.006667            16.013333
std     43.445368         2.703457             8.157828
min    101.000000         1.000000             1.000000
25%    138.250000         4.000000            10.000000
50%    175.500000         6.000000            16.000000
75%    212.750000         8.000000            23.000000
max    250.000000        10.000000            30.000000

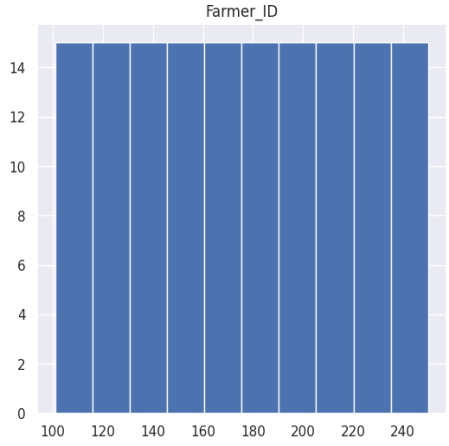
       Previous_Loan_Default        Income  Yield(National Standard)  \
count             150.000000    150.000000                150.000000
mean                0.520000  33857.866667               2584.653384
std                 0.501274   8412.362456               2156.030562
min                 0.000000  20255.000000                 52.780000
25%                 0.000000  26826.750000               1047.000000
50%                 1.000000  33736.500000               1697.000000
75%                 1.000000  40408.750000               5065.147500
max                 1.000000  49567.000000               6739.000000

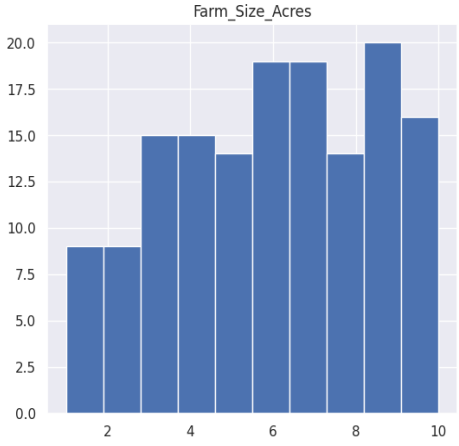
              CVI    TotalYield  Loan_Default
count  150.000000    150.000000    150.000000
mean     0.474333  14997.452018      0.460000
std      0.035886  15100.591317      0.500067
min      0.420000    116.740000      0.000000
25%      0.440000   3248.500000      0.000000
```

```
# Generate histograms for each numerical column
data.hist(figsize=(22, 20))

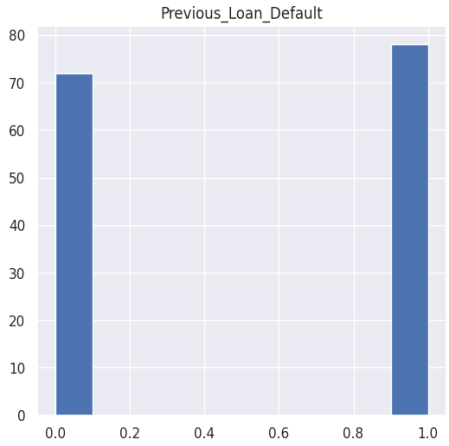
# Display the plot
plt.show()
```

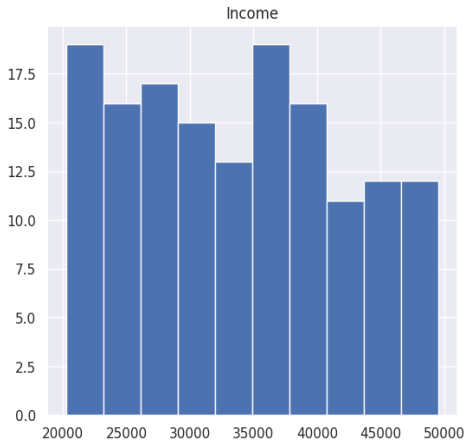
Step 2: Data Exploration

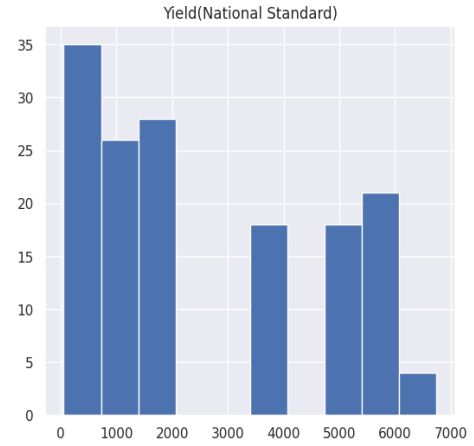


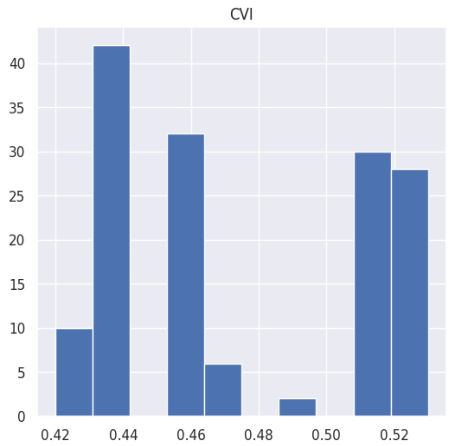


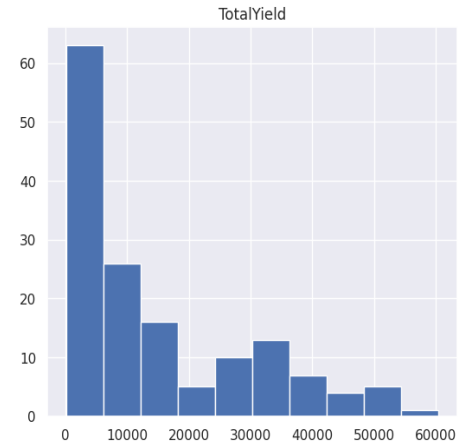


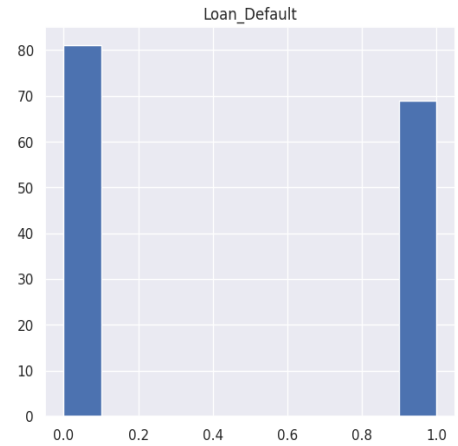












Visualization of Data Points and their values.

```
# Count the occurrences of each unique value in the "Loan_Default" column
loan_default_counts = data["Loan_Default"].value_counts()

# Print the counts
print(loan_default_counts)
```

```
Loan_Default
0    81
1    69
Name: count, dtype: int64
```

```
# Selecting only the numerical columns for correlation calculation
numerical_data = data[['Farm_Size_Acres', 'Years_of_Experience', 'Previous_Loan_Default', 'Income', 'Yield(National Standard)', 'CVI', 'T

# Create a correlation matrix
corr_matrix = numerical_data.corr()

# Selecting only the numerical columns for correlation calculation
numerical_data = data[['Farm_Size_Acres', 'Years_of_Experience', 'Previous_Loan_Default', 'Income', 'Yield(National Standard)', 'CVI', 'T

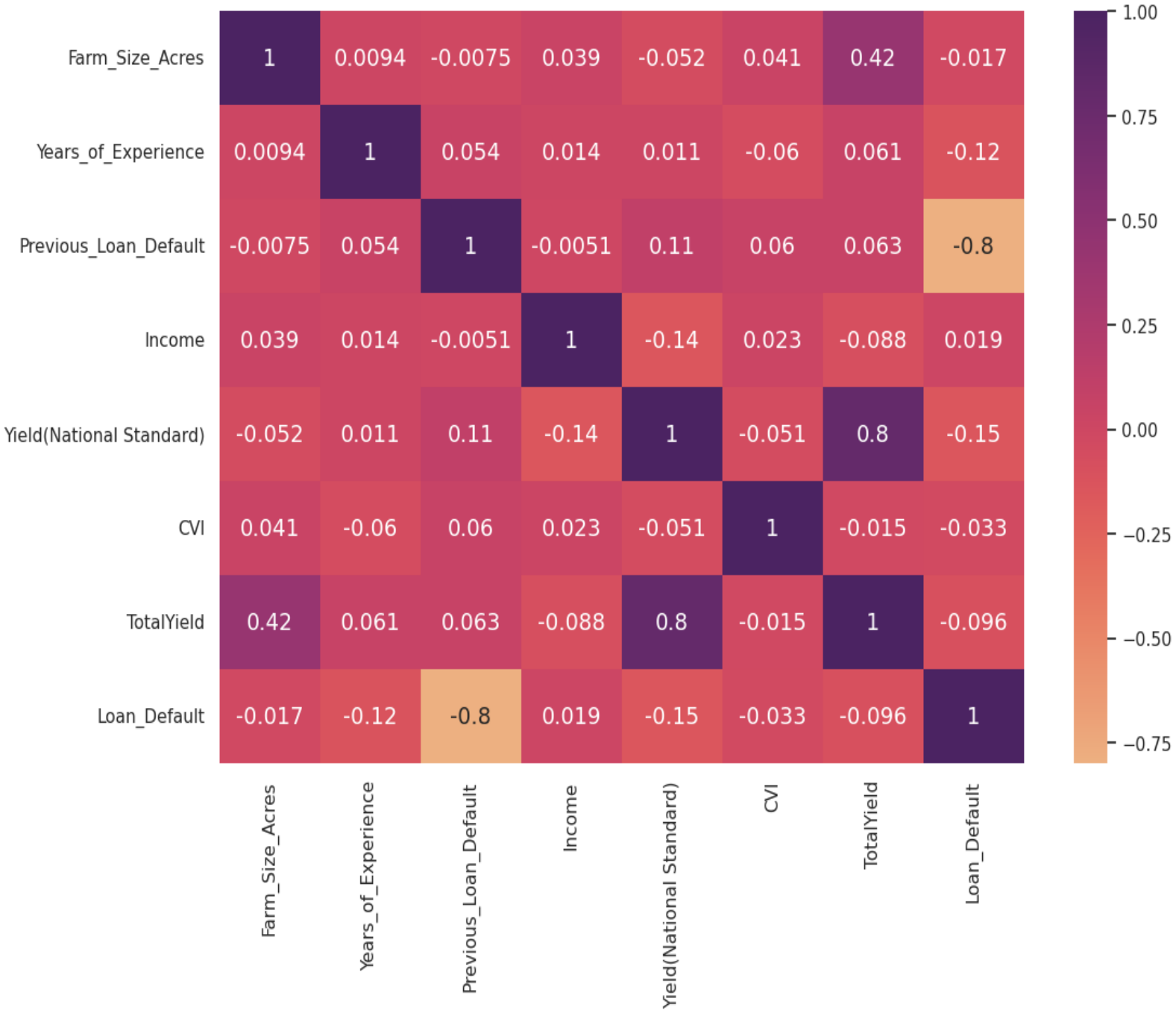
# Create a correlation matrix
corr_matrix = numerical_data.corr()

# Print the correlation matrix
print(corr_matrix)


# Create a figure and axis with the specified size
fig, ax = plt.subplots(figsize=(12, 8))

# Create a heatmap of the correlation matrix
corr_heatmap = sns.heatmap(corr_matrix, cmap="flare", annot=True, ax=ax, annot_kws={"size": 14})

# Display the plot
plt.show()
```



Correlation Analysis Through HeatMap

Findings from Correlations Matrix Analysis and Their Interpretation:

1. **Farm Size and Total Yield**: There is a moderately positive correlation (0.42) between the size of the farm (measured in acres) and the total yield of the crop. This suggests that larger farms tend to produce higher yields.

2. **Years of Experience and Loan Default**: There is a weak negative correlation (-0.12) between the number of years of farming experience and the likelihood of loan default. This suggests that farmers with more experience are slightly less likely to default on loans.

3. **Previous Loan Default and Loan Default**: There is a strong negative correlation (-0.80) between previous loan default and current loan default. This indicates a significant relationship, suggesting that farmers who have defaulted on previous loans are much more likely to default on current loans.

4. **Yield and Income**: There is a moderate negative correlation (-0.14) between the yield of the crop and the income generated from farming. This suggests that higher yields may not always directly translate to higher income, possibly due to factors such as market prices or production costs.

5. **Yield and Total Yield**: There is a strong positive correlation (0.80) between the yield of the crop and the total yield. This indicates that higher yields at the individual crop level are associated with higher total yields across all crops.

6. **Previous Loan Default and Yield**: There is a weak positive correlation (0.11) between previous loan default and the yield of the crop. This suggests a slight association between previous loan default and higher crop yields.

7. **Years of Experience and CVI**: There is a weak negative correlation (-0.06) between years of farming experience and the Crop Value Index (CVI). This indicates a slight tendency for farmers with more experience to have lower CVI values.

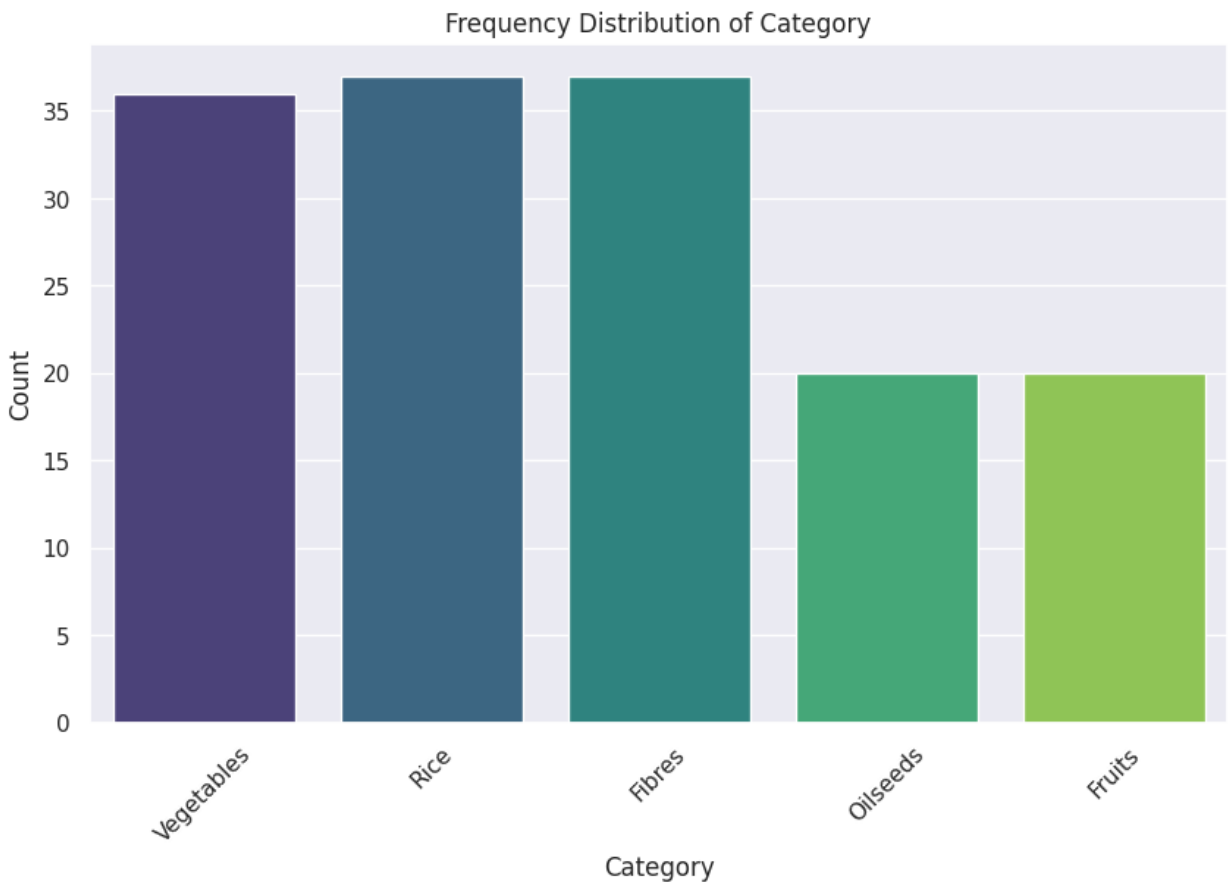
8. **Farm Size and Yield**: There is a weak negative correlation (-0.05) between farm size and the yield of the crop. This suggests a slight tendency for larger farms to have lower crop yields.

```python
# Frequency distribution of 'Category'
category_counts = data['Category'].value_counts()
print("Category counts:")
print(category_counts)
print()

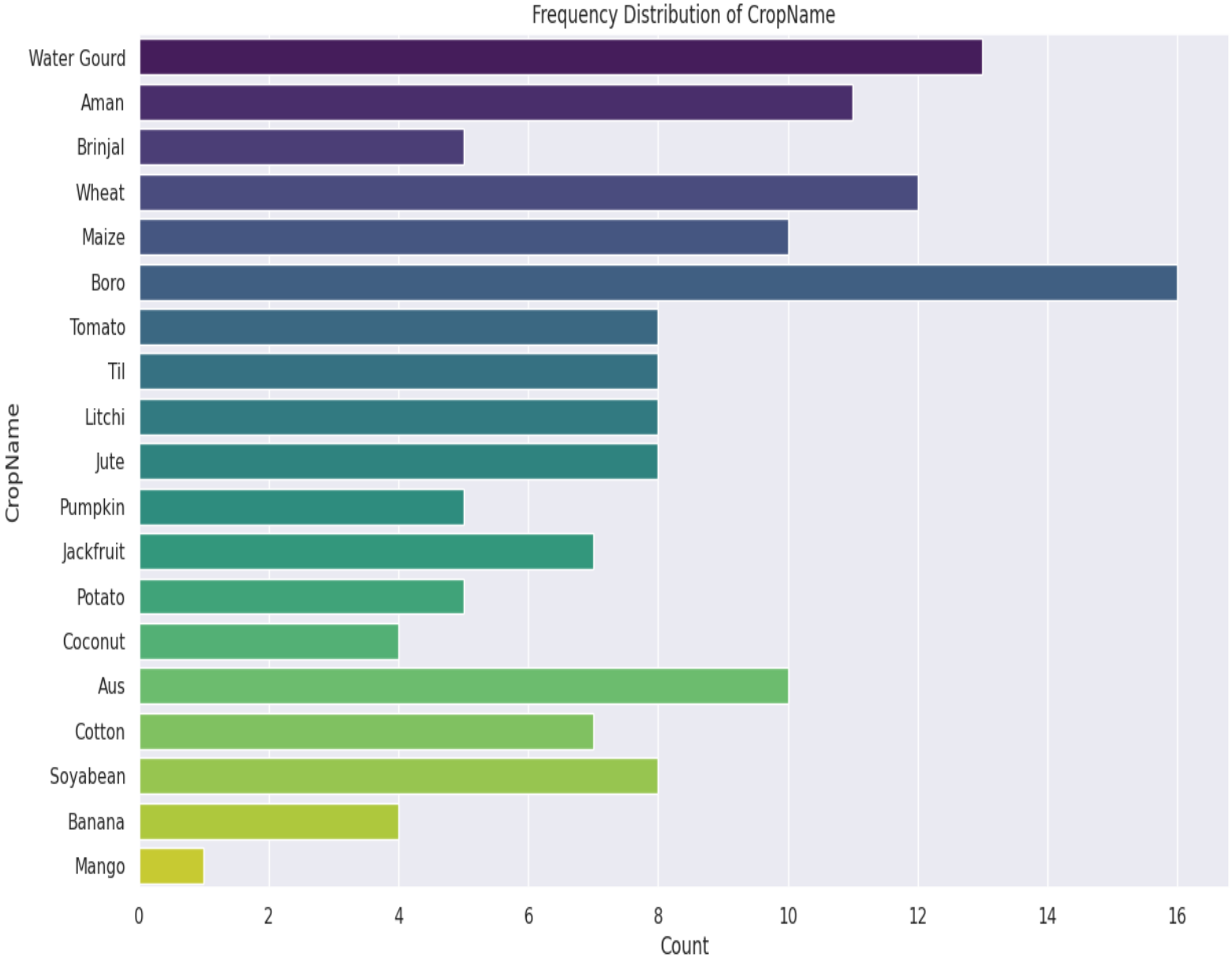
# Visualize the frequency distribution of 'Category' using a bar plot
plt.figure(figsize=(10, 6))
sns.countplot(data=data, x='Category', palette='viridis')
plt.title('Frequency Distribution of Category')
plt.xlabel('Category')
plt.ylabel('Count')
plt.xticks(rotation=45)
plt.show()

# Frequency distribution of 'CropName'
cropname_counts = data['CropName'].value_counts()
print("CropName counts:")
print(cropname_counts)
print()

# Visualize the frequency distribution of 'CropName' using a bar plot
plt.figure(figsize=(14, 8))
sns.countplot(data=data, y='CropName', palette='viridis')
plt.title('Frequency Distribution of CropName')
plt.xlabel('Count')
plt.ylabel('CropName')
plt.show()
```




```
CropName counts:
CropName
Boro           16
Water Gourd    13
Wheat          12
Aman           11
Maize          10
Aus            10
Litchi          8
Soyabean        8
Jute            8
Til             8
Tomato          8
Jackfruit       7
Cotton          7
Pumpkin         5
Potato          5
Brinjal         5
Coconut         4
Banana          4
Mango           1
Name: count, dtype: int64
```


Frequency Distribution of CropName
Water Gourd
Aman
Brinjal
Wheat
Maize
Boro
Tomato
Til
Litchi
Jute
Pumpkin
Jackfruit
Potato
Coconut
Aus
Cotton
Soyabean
Banana
Mango
CropName
0
2
4
6
8
10
12
14
16
Count

```python
from scipy.stats import chi2_contingency

# Cross-tabulation between 'Category' and 'Loan_Default'
category_loan_cross = pd.crosstab(index=data['Category'], columns=data['Loan_Default'])
print("Cross-tabulation between Category and Loan_Default:")
print(category_loan_cross)
print()

# Chi-square test of independence between 'Category' and 'Loan_Default'
chi2, p, _, _ = chi2_contingency(category_loan_cross)
print("Chi-square test between Category and Loan_Default:")
print("Chi-square statistic:", chi2)
print("p-value:", p)
print()

# Cross-tabulation between 'CropName' and 'Loan_Default'
cropname_loan_cross = pd.crosstab(index=data['CropName'], columns=data['Loan_Default'])
print("Cross-tabulation between CropName and Loan_Default:")
print(cropname_loan_cross)
print()

# Chi-square test of independence between 'CropName' and 'Loan_Default'
chi2, p, _, _ = chi2_contingency(cropname_loan_cross)
print("Chi-square test between CropName and Loan_Default:")
print("Chi-square statistic:", chi2)
print("p-value:", p)
```

```
Cross-tabulation between Category and Loan_Default:
Loan_Default   0   1
Category
Fibres        18  19
Fruits         7  13
Oilseeds      15   5
Rice          15  22
Vegetables    26  10

Chi-square test between Category and Loan_Default:
Chi-square statistic: 14.394587824539514
p-value: 0.006136566941988282

Cross-tabulation between CropName and Loan_Default:
Loan_Default  0  1
CropName
Aman          4  7
Aus           2  8
Banana        1  3
Boro          9  7
Brinjal       3  2
Coconut       1  3
Cotton        3  4
Jackfruit     3  4
Jute          5  3
Litchi        3  5
Maize         3  7
Mango         0  1
Potato        5  0
Pumpkin       3  2
Soyabean      6  2
Til           8  0
Tomato        6  2
Water Gourd   9  4
Wheat         7  5

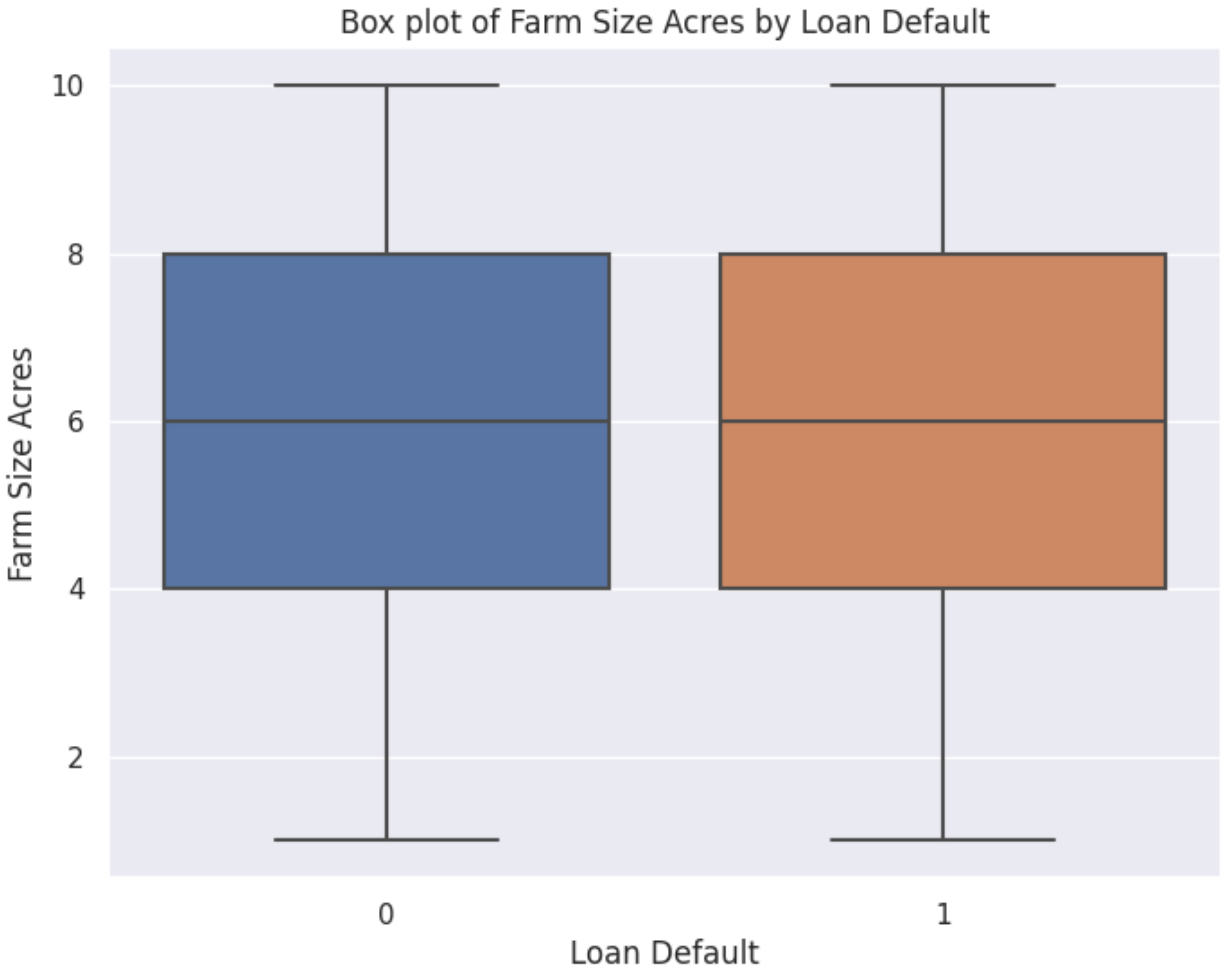
Chi-square test between CropName and Loan_Default:
Chi-square statistic: 29.43871413074311
p-value: 0.043274023404935386
```

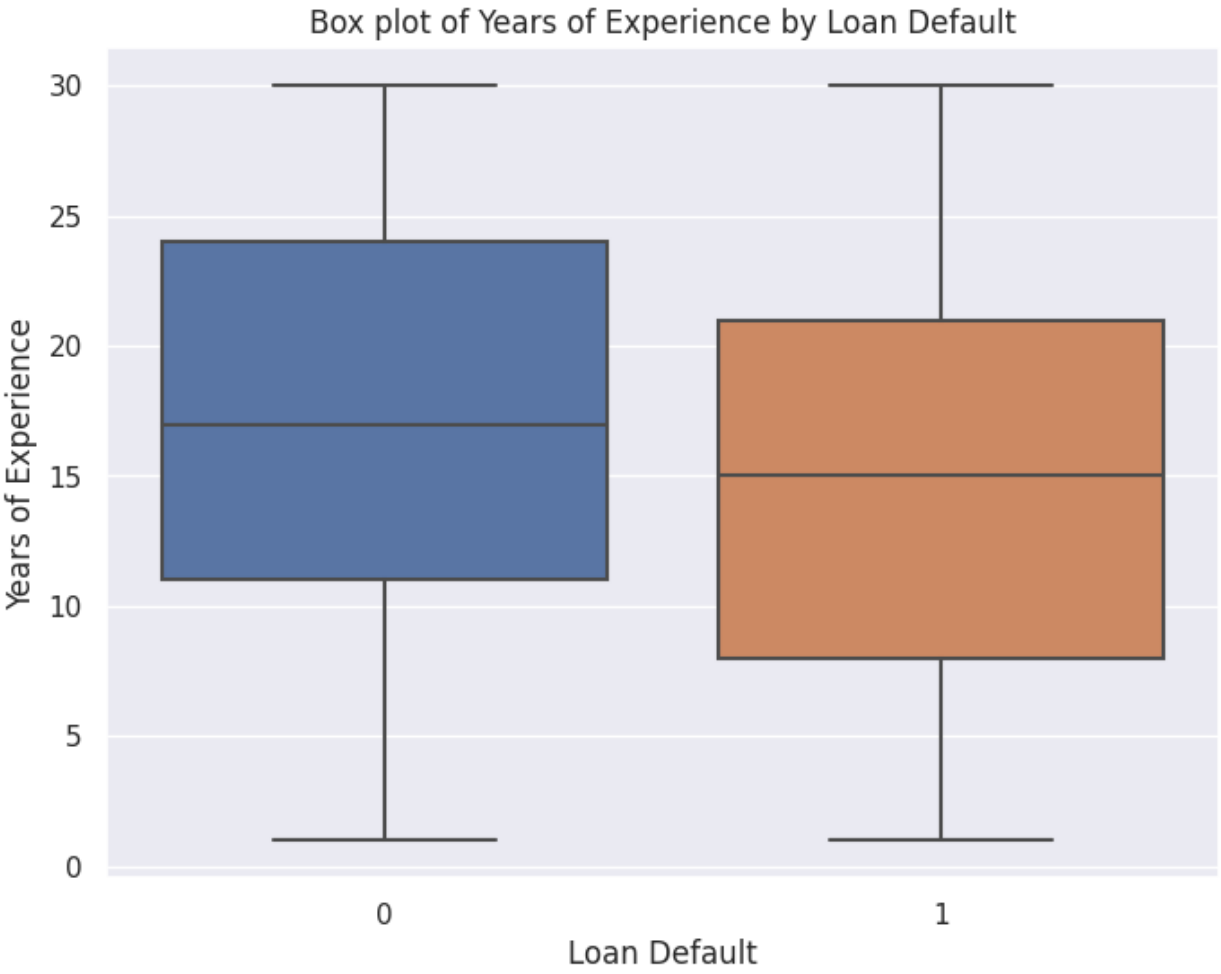
```
import matplotlib.pyplot as plt
import seaborn as sns

# Box plot for 'Loan_Default' and 'Farm_Size_Acres'
plt.figure(figsize=(8, 6))
sns.boxplot(x='Loan_Default', y='Farm_Size_Acres', data=data)
plt.title('Box plot of Farm Size Acres by Loan Default')
plt.xlabel('Loan Default')
plt.ylabel('Farm Size Acres')
plt.show()

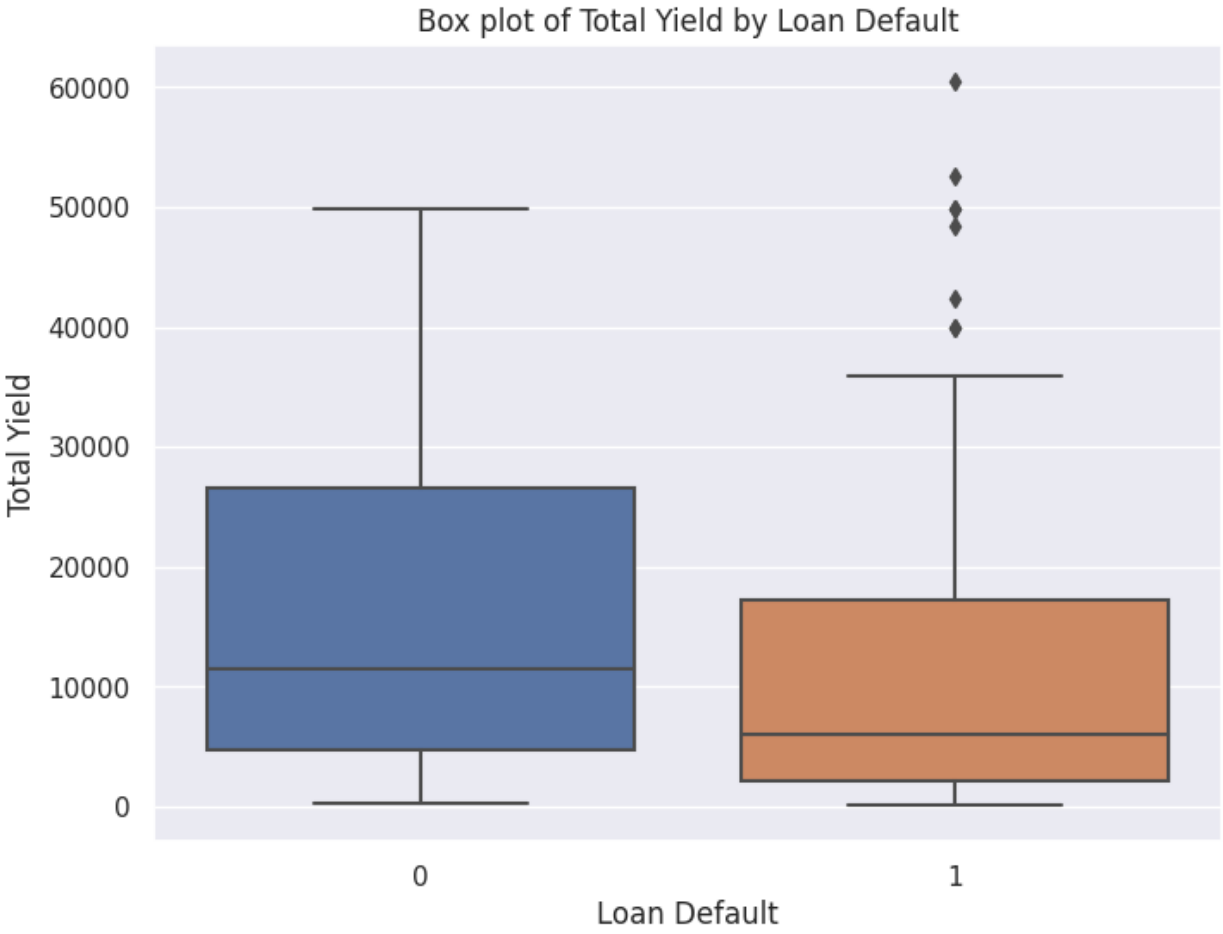
# Box plot for 'Loan_Default' and 'Years_of_Experience'
plt.figure(figsize=(8, 6))
sns.boxplot(x='Loan_Default', y='Years_of_Experience', data=data)
plt.title('Box plot of Years of Experience by Loan Default')
plt.xlabel('Loan Default')
plt.ylabel('Years of Experience')
plt.show()

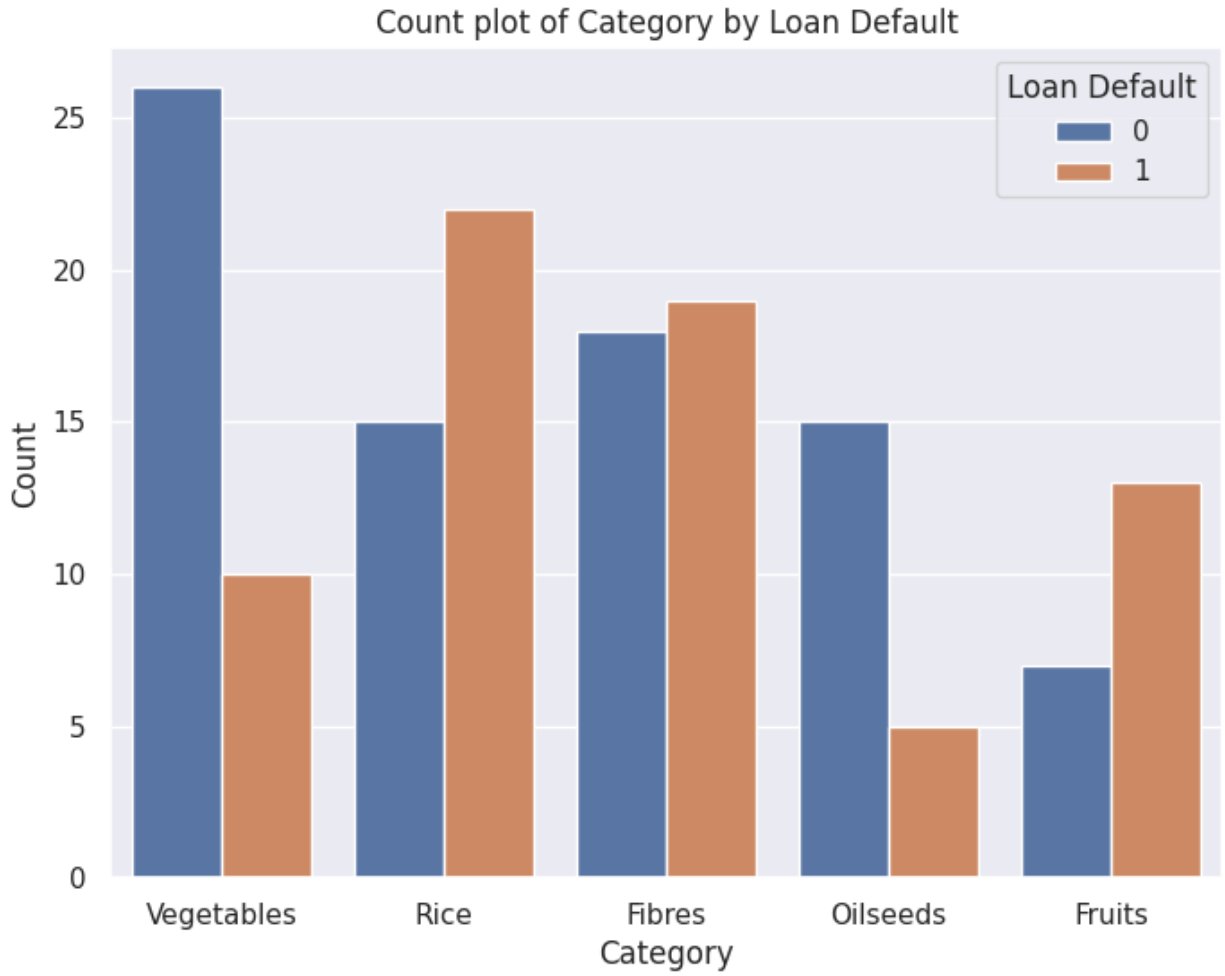
# Box plot for 'Loan_Default' and 'Income'
plt.figure(figsize=(8, 6))
sns.boxplot(x='Loan_Default', y='Income', data=data)
plt.title('Box plot of Income by Loan Default')
plt.xlabel('Loan Default')
plt.ylabel('Income')
plt.show()
```


Box plot of Farm Size Acres by Loan Default
Farm Size Acres
10
8
6
4
2
0
1
Loan Default


Box plot of Years of Experience by Loan Default
Years of Experience
30
25
20
15
10
5
0
0
1
Loan Default


Box plot of Income by Loan Default
50000
45000
40000
35000
30000
25000
20000
Income
0
1
Loan Default


Box plot of Total Yield by Loan Default
60000
50000
40000
30000
20000
10000
0
Total Yield
0
1
Loan Default

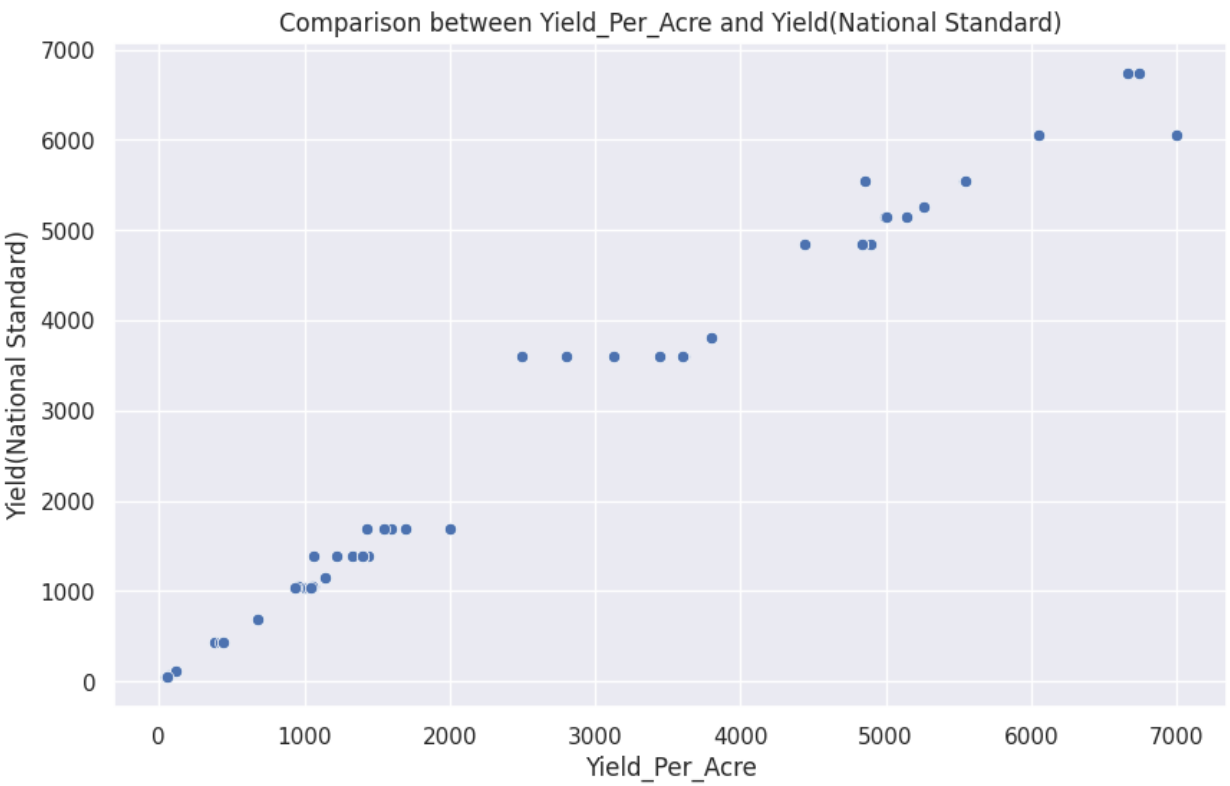


Step 4: Feature Extraction

```
# Perform feature extraction to create a new feature 'Yield_Per_Acre'
data['Yield_Per_Acre'] = data['TotalYield'] / data['Farm_Size_Acres']

# Display the first few rows of the DataFrame to verify the new feature
print(data.head())

# Visualize the comparison between 'Yield_Per_Acre' and 'Yield(National Standard
plt.figure(figsize=(10, 6))
sns.scatterplot(x='Yield_Per_Acre', y='Yield(National Standard)', data=data)
plt.title('Comparison between Yield_Per_Acre and Yield(National Standard)')
plt.xlabel('Yield_Per_Acre')
plt.ylabel('Yield(National Standard)')
plt.show()
```


Comparison between Yield_Per_Acre and Yield(National Standard)
Yield(National Standard)
Yield_Per_Acre
0
1000
2000
3000
4000
5000
6000
7000

```python
# Calculate Income_Per_Year
data['Income_Per_Year'] = data['Income'] / data['Years_of_Experience']

# Calculate Yield_Ratio
data['Yield_Ratio'] = data['TotalYield'] / data['Yield(National Standard)']



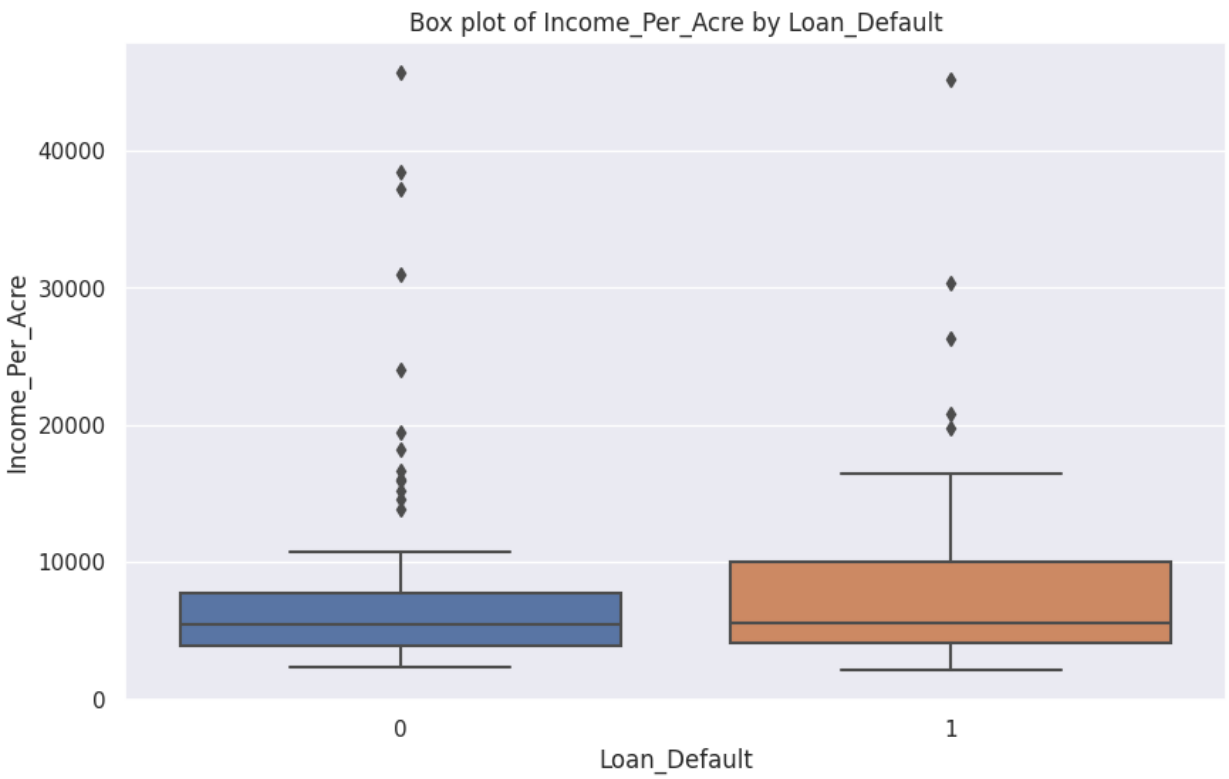
# Display the first few rows of the DataFrame to verify the new features
print(data.head())
```

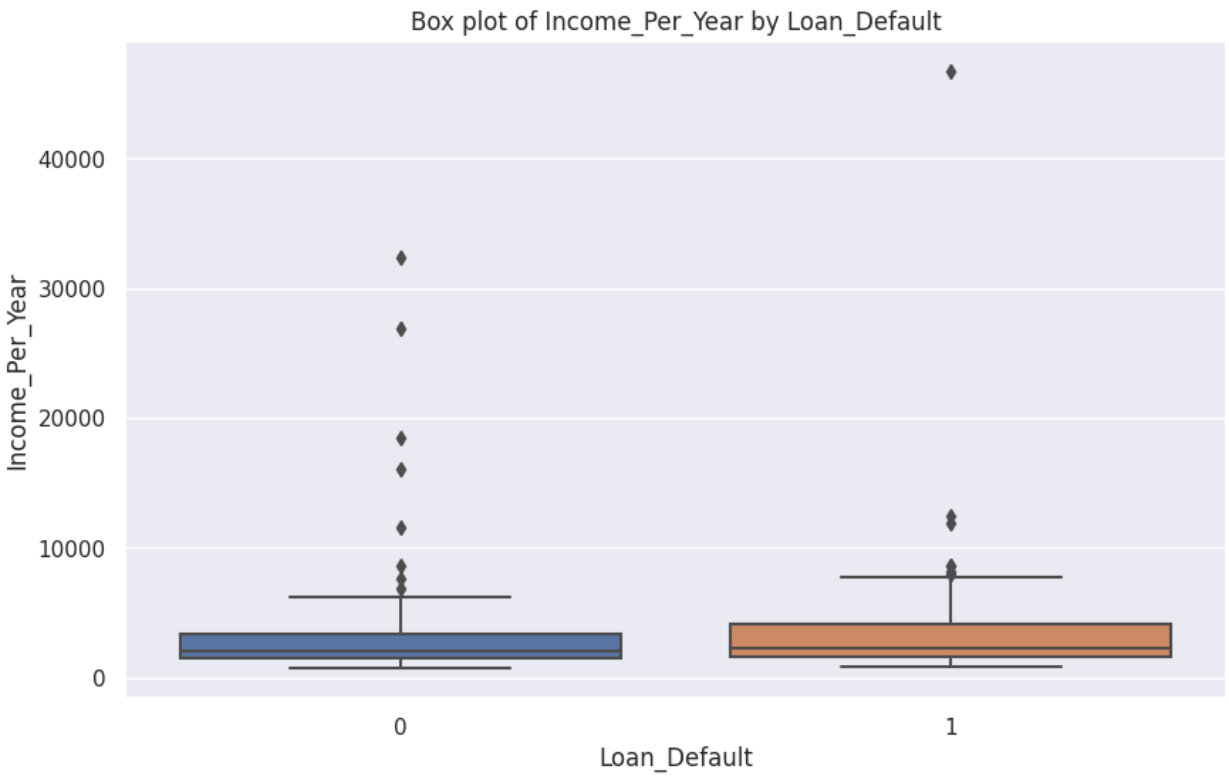
```
   Farmer_ID  Farm_Size_Acres    Category     CropName  Years_of_Experience  \
0        101                3  Vegetables  Water Gourd                   18
1        102                3        Rice         Aman                    8
2        103               10  Vegetables      Brinjal                   19
3        104                2      Fibres        Wheat                   10
4        105                6      Fibres        Maize                   23

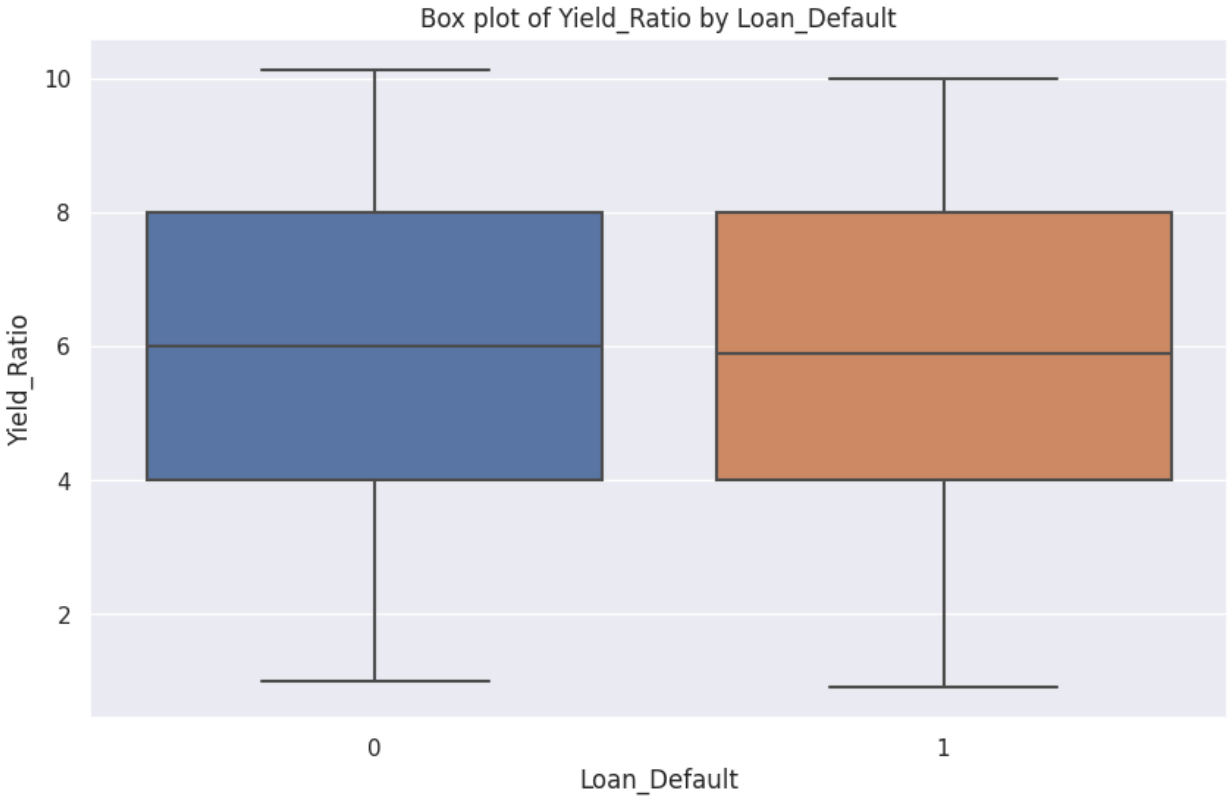
   Previous_Loan_Default  Income  Location  Yield(National Standard)   CVI  \
0                      0   31527  Faridpur                   5547.87  0.46
1                      0   40152   Rangpur                   1058.00  0.51
2                      0   28392   Rangpur                   4838.16  0.51
3                      1   36435  Barishal                   1395.00  0.53
4                      1   41728     Dhaka                   3605.00  0.44

   TotalYield  Loan_Default  Yield_Per_Acre  Income_Per_Acre  Income_Per_Year  \
0     16600.0             1     5533.333333     10509.000000      1751.500000
1      3200.0             1     1066.666667     13384.000000      5019.000000
2     48999.0             0     4899.900000      2839.200000      1494.315789
3      2880.0             0     1440.000000     18217.500000      3643.500000
4     21640.0             0     3606.666667      6954.666667      1814.260870

   Yield_Ratio
0     2.992139
1     3.024575
2    10.127610
3     2.064516
4     6.002774
```


Box plot of Income_Per_Acre by Loan_Default
Income_Per_Acre
0
10000
20000
30000
40000
0
1
Loan_Default


Box plot of Income_Per_Year by Loan_Default
Income_Per_Year
0
10000
20000
30000
40000
0
1
Loan_Default



Step 5: Data Cleaning

```
from sklearn.preprocessing import LabelEncoder

# Initialize LabelEncoder
label_encoder = LabelEncoder()

# Encode the 'Category' variable
data['Category_Encoded'] = label_encoder.fit_transform(data['Category'])

# Encode the 'CropName' variable
data['CropName_Encoded'] = label_encoder.fit_transform(data['CropName'])

# Display the first few rows of the DataFrame with encoded variables
print(data.head())
```

	Farmer_ID	Farm_Size_Acres	Category	CropName	Years_of_Experience	\
0	101	3	Vegetables	Water Gourd	18	
1	102	3	Rice	Aman	8	
2	103	10	Vegetables	Brinjal	19	
3	104	2	Fibres	Wheat	10	
4	105	6	Fibres	Maize	23	

	Previous_Loan_Default	Income	Location	Yield(National Standard)	CVI	\
0	0	31527	Faridpur	5547.87	0.46	
1	0	40152	Rangpur	1058.00	0.51	
2	0	28392	Rangpur	4838.16	0.51	
3	1	36435	Barishal	1395.00	0.53	
4	1	41728	Dhaka	3605.00	0.44	

	TotalYield	Loan_Default	Yield_Per_Acre	Income_Per_Acre	Income_Per_Year	\
0	16600.0	1	5533.333333	10509.000000	1751.500000	
1	3200.0	1	1066.666667	13384.000000	5019.000000	
2	48999.0	0	4899.900000	2839.200000	1494.315789	
3	2880.0	0	1440.000000	18217.500000	3643.500000	
4	21640.0	0	3606.666667	6954.666667	1814.260870	

	Yield_Ratio	Category_Encoded	CropName_Encoded
0	2.992139	4	17
1	3.024575	3	0
2	10.127610	4	4
3	2.064516	0	18
4	6.002774	0	10

```
from sklearn.preprocessing import OneHotEncoder

# Perform one-hot encoding for the 'Location' variable
encoder = OneHotEncoder(sparse=False)
location_encoded = encoder.fit_transform(data[['Location']])

# Get the feature names for the one-hot encoded 'Location' variable
location_categories = encoder.categories_[0]
location_encoded_df = pd.DataFrame(location_encoded, columns=[f'Location_{cat}' for cat in location_categories])

# Concatenate the original DataFrame with the one-hot encoded 'Location' variable
data_encoded = pd.concat([data, location_encoded_df], axis=1)

# Remove the original categorical variables from the DataFrame
data_encoded.drop(['Category', 'CropName', 'Location'], axis=1, inplace=True)

# Display the first few rows of the encoded DataFrame
print(data_encoded.head())
```

	Farmer_ID	Farm_Size_Acres	Years_of_Experience	Previous_Loan_Default	\
0	101	3	18	0	
1	102	3	8	0	
2	103	10	19	0	
3	104	2	10	1	
4	105	6	23	1	

	Income	Yield(National Standard)	CVI	TotalYield	Loan_Default	\
0	31527	5547.87	0.46	16600.0	1	
1	40152	1058.00	0.51	3200.0	1	
2	28392	4838.16	0.51	48999.0	0	
3	36435	1395.00	0.53	2880.0	0	
4	41728	3605.00	0.44	21640.0	0	

	Yield_Per_Acre	...	Location_Faridpur	Location_Gazipur	Location_Khulna	\
0	5533.333333	...	1.0	0.0	0.0	
1	1066.666667	...	0.0	0.0	0.0	
2	4899.900000	...	0.0	0.0	0.0	
3	1440.000000	...	0.0	0.0	0.0	
4	3606.666667	...	0.0	0.0	0.0	

	Location_Moulvibazar	Location_Mymensingh	Location_Narayanganj	\
0	0.0	0.0	0.0	
1	0.0	0.0	0.0	
2	0.0	0.0	0.0	
3	0.0	0.0	0.0	
4	0.0	0.0	0.0	

	Location_Noakhali	Location_Rajshahi	Location_Rangpur	Location_Sylhet
0	0.0	0.0	0.0	0.0
1	0.0	0.0	1.0	0.0
2	0.0	0.0	1.0	0.0
3	0.0	0.0	0.0	0.0
4	0.0	0.0	0.0	0.0

[5 rows x 30 columns]

Step 6: Model Development

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Define features (X) and target variable (y)
X = data_encoded.drop('Loan_Default', axis=1)
y = data_encoded['Loan_Default']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Initialize and fit the random forest classifier
rf_classifier = RandomForestClassifier(n_estimators=100, random_state=42)
rf_classifier.fit(X_train, y_train)

# Predict on the testing set
y_pred = rf_classifier.predict(X_test)
```

Step 7: Model Assesment

```
from sklearn.metrics import precision_score, recall_score, f1_score, roc_auc_score, roc_curve

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

# Calculate precision
precision = precision_score(y_test, y_pred)
print("Precision:", precision)

# Calculate recall
recall = recall_score(y_test, y_pred)
print("Recall:", recall)

# Calculate F1-score
f1 = f1_score(y_test, y_pred)
print("F1-score:", f1)

# Calculate ROC-AUC
y_prob = rf_classifier.predict_proba(X_test)[:, 1]
roc_auc = roc_auc_score(y_test, y_prob)
print("ROC-AUC:", roc_auc)
```

```
Accuracy: 0.9111111111111111
Precision: 0.9473684210526315
Recall: 0.8571428571428571
F1-score: 0.9
ROC-AUC: 0.9494047619047619
```

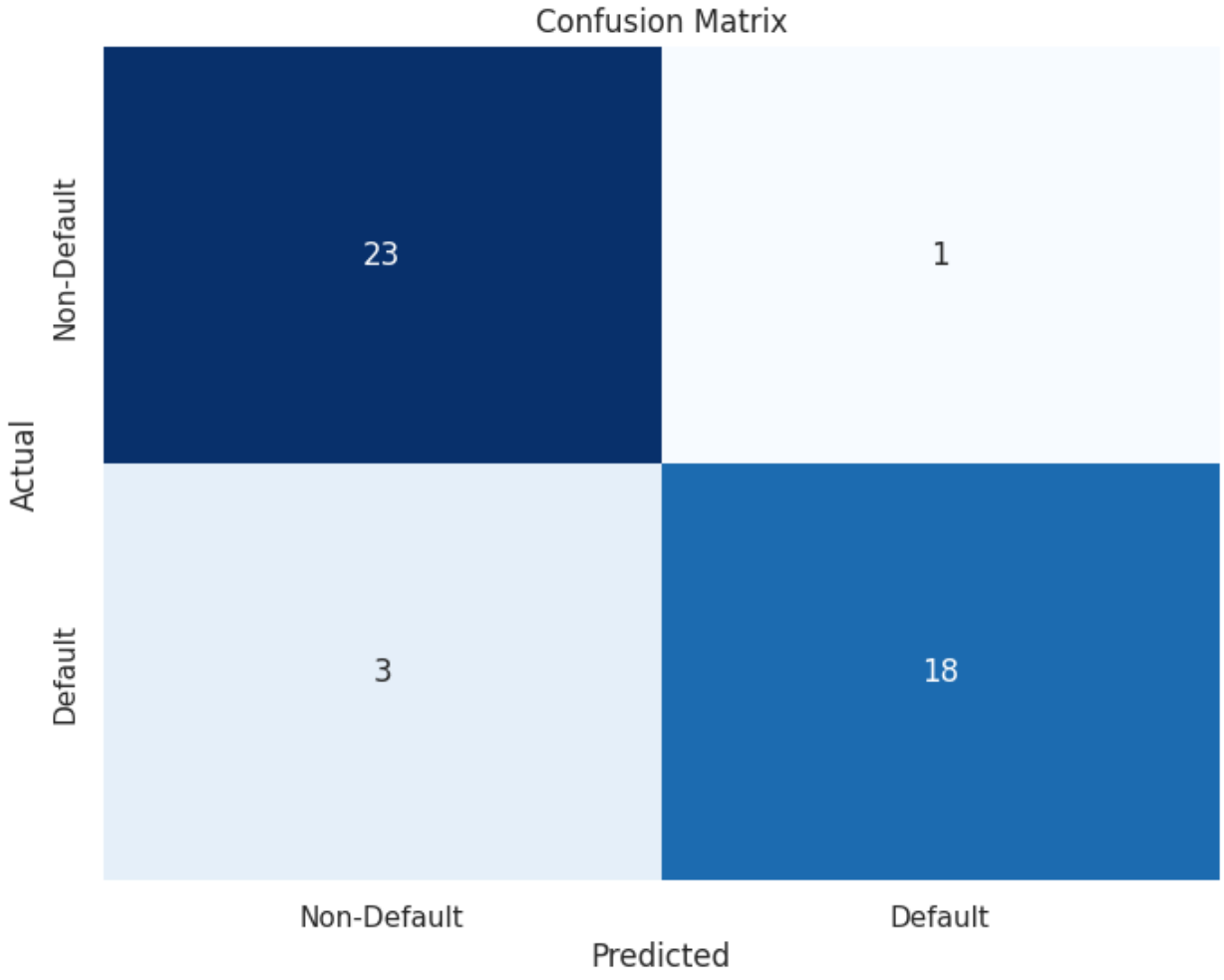
Accuracy: 0.9111111111111111
Precision: 0.9473684210526315
Recall: 0.8571428571428571
F1-score: 0.9
ROC-AUC: 0.9494047619047619

Models Accuracy is 91.111%

```python
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

# Compute confusion matrix
conf_matrix = confusion_matrix(y_test, y_pred)

# Plot confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues', cbar=False,
            xticklabels=['Non-Default', 'Default'], yticklabels=['Non-Default', 'Default'])
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()
```



Confusion Matrix

Summary of the Task-2

The project aimed to analyze a dataset related to agriculture and predict loan default based on various features.

It involved data preprocessing, exploratory data analysis (EDA), feature engineering, model training, and evaluation.

Data Preprocessing:

- The dataset contained information about farmers, including features like farm size, crop category, years of experience, income, location, yield, etc.
- Data preprocessing involved handling missing values, encoding categorical variables, feature scaling, and splitting the data into training and test sets.

Exploratory Data Analysis (EDA):

- EDA was performed to understand the distribution and relationships between different variables.
- Visualizations such as histograms, box plots, and heatmaps were used to analyze numerical variables and correlations.
- Categorical variables were analyzed using count plots and chi-square tests to understand their relationship with the target variable (loan default).

Feature Engineering:

- New features were created to enhance the predictive power of the model, such as income per acre, yield per acre, income per year of experience, yield ratio, etc.
- One-hot encoding was applied to categorical variables to convert them into numerical format suitable for modeling.

Model Training and Evaluation:

- A random forest classifier was chosen as the predictive model due to its ability to handle complex relationships and provide feature importances.
- The model was trained on the training dataset and evaluated using various metrics such as accuracy, precision, recall, F1-score, and ROC-AUC.
- Confusion matrix and visualization were used to understand the model's performance in predicting loan default.

Key Findings:

- The dataset showed significant variation in features like farm size, income, and yield across different locations and crop categories.
- The random forest classifier achieved a certain level of accuracy in predicting loan default based on the available features.
- Features such as farm size, income, previous loan default, and yield had significant importance in predicting loan default.
- The model's performance could be further improved by incorporating additional features or using more sophisticated modeling techniques.

Recommendations:

- Based on the findings, it is recommended to collect more data on factors influencing loan default, such as weather conditions, government policies, market trends, etc.
- Advanced modeling techniques like gradient boosting machines or neural networks could be explored to improve predictive performance.
- Regular monitoring and updating of the model will be necessary to ensure its effectiveness in predicting loan default in the agricultural sector.